实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**数据库系统**			学期	**2025 年秋季学期**
任课教师	刘艳艳	完成日期	2025/11/20	上机课时间	2025/10/29
实 验 名 称	学生教务管理系统的数据库设计、查询、更新及原型系统实现				

一、实验要求（10%）

实现教务系统数据库中数据表的设计，使用商用数据库 SQLServer 或 MySQL 实现所设计的数据库，实现数据的存储、查询、更新、删除等，并基于所设计的数据库实现一个“学生教务管理系统”，具体需包含以下内容：

（1）设计教务系统数据库中的数据表，可直接参考书上第二章给出的学生表、课程表和选课表，用 SQL 语句建立学生教务数据库，并对每张表录入实验数据，对数据库中所讲的表进行截图附在实验报告中。

（2）结合教材所学的 SQL 语句，进行实际操作，练习各类查询、更新、修、改、删除等语句，将其中自己认为最重要的 20 个 SQL 查询、更新、修改、删除等操作写到实验报告中，并对各 SQL 语句及其运行结果的屏幕进行截图附在实验报告中，必须包括单表查询、多表查询、带函数表达式的查询、相关子查询、不相关子查询、集函数查询、表达“全部”语义的查询（如查询选修全部课程的学生）、表达“至少”语义的查询（如查询至少选修了小明同学选修的全部课程的学生）、带 Exist 的子查询、使用子查询进行批量插入删除以及修改的语句、外连接查询等。

（3）使用.net 或 Java 等，基于所设计的数据库实现一个学生教务管理系统，应包括对数据库中数据的查询、添加、删除、修改等功能，界面友好，B/S 或 C/S 架构不限，将各界面演示及核心代码附在实验报告中。

二、实验内容及步骤（80%）

1、创建数据库

```
CREATE DATABASE lab3;

USE lab3;
```

2、设计教务系统数据库中的数据表并插入数据

（1）student 表

建表语句如下：

```
CREATE TABLE student
(
    SNO         CHAR(8)     NOT NULL    COMMENT "学号" PRIMARY KEY ,
    SNAME       VARCHAR(20) NOT NULL    COMMENT "姓名",
    SSEX        CHAR        NOT NULL    COMMENT "性别",
    SBIRTHDATE  DATE        NOT NULL    COMMENT "出生日期",
    SMAJOR      VARCHAR(20) NOT NULL    COMMENT "主修专业"
)COMMENT "学生";
```

插入语句如下：

```
INSERT INTO student
VALUES ('20180001','李勇','男','2000-3-8','信息安全'),
       ('20180002','刘晨','女','1999-9-1','计算机科学与技术'),
       ('20180003','王敏','女','2001-8-1','计算机科学与技术'),
       ('20180004','张立','男','2000-1-8','计算机科学与技术'),
       ('20180005','陈新奇','男','2001-11-1','信息管理与信息系统'),
       ('20180006','赵明','男','2000-6-12','数据科学与大数据技术'),
       ('20180007','王佳佳','女','2001-12-7','数据科学与大数据技术');
```

最终表格如下：

	SNO	SNAME	SSEX	SBIRTHDATE	SMAJOR
1	20180001	李勇	男	2000-03-08	信息安全
2	20180002	刘晨	女	1999-09-01	计算机科学与技术
3	20180003	王敏	女	2001-08-01	计算机科学与技术
4	20180004	张立	男	2000-01-08	计算机科学与技术
5	20180005	陈新奇	男	2001-11-01	信息管理与信息系统
6	20180006	赵明	男	2000-06-12	数据科学与大数据技
7	20180007	王佳佳	女	2001-12-07	数据科学与大数据技

（2）course 表

建表语句如下：

```
CREATE TABLE COURSE
(
    CNO         CHAR(5)     NOT NULL    COMMENT "课程号" PRIMARY KEY ,
    CNAME       VARCHAR(20) NOT NULL    COMMENT "课程名" UNIQUE ,
    CCREDIT     INT         NOT NULL    COMMENT "学分",
    CPNO        CHAR(5)     COMMENT "先修课",
    FOREIGN KEY (CPNO) REFERENCES COURSE(CNO) ON UPDATE CASCADE
)COMMENT "课程";
```

插入语句如下：

```
INSERT INTO COURSE
VALUES ('81001','程序设计基础与C语言','4',NULL),
       ('81002','数据结构','4','81001'),
       ('81003','数据库系统概论','4','81002'),
       ('81004','信息系统概论','4','81003'),
       ('81005','操作系统','4','81001'),
       ('81006','Python语言','3','81002'),
       ('81007','离散数学','4',NULL),
       ('81008','大数据技术概论','4','81003');
```

最终表格如下：

	CNO	CNAME	CCREDIT	CPNO
1	81001	程序设计基础与C	4	<null>
2	81002	数据结构	4	81001
3	81003	数据库系统概论	4	81002
4	81004	信息系统概论	4	81003
5	81005	操作系统	4	81001
6	81006	Python语言	3	81002
7	81007	离散数学	4	<null>
8	81008	大数据技术概论	4	81003

（3）SC 表

建表语句如下：

```
CREATE TABLE SC
(
    SNO             CHAR(8)     NOT NULL    COMMENT "学号",
    CNO             CHAR(5)     NOT NULL    COMMENT "课程号",
    GRADE           SMALLINT    NOT NULL    COMMENT "成绩",
    SEMESTER        CHAR(5)     NOT NULL    COMMENT "开课学期",
    TEACHINGCLASS   CHAR(8)     NOT NULL    COMMENT "教学班",
    PRIMARY KEY (SNO, CNO),
    FOREIGN KEY SC(SNO) REFERENCES student(SNO) ON UPDATE CASCADE ,
    FOREIGN KEY SC(CNO) REFERENCES COURSE(CNO) ON UPDATE CASCADE
)COMMENT "选修";
```

插入语句如下：

```sql
INSERT INTO SC
VALUES ('20180001','81001','85','20192','81001-01'),
       ('20180001','81002','96','20201','81002-01'),
       ('20180001','81003','87','20202','81003-01'),
       ('20180002','81001','80','20192','81001-02'),
       ('20180002','81002','98','20201','81002-01'),
       ('20180002','81003','71','20202','81003-02'),
       ('20180003','81001','81','20192','81001-01'),
       ('20180003','81002','76','20201','81002-02'),
       ('20180004','81001','56','20192','81001-02'),
       ('20180004','81002','97','20201','81002-02'),
       ('20180005','81003','68','20202','81003-01');
```

最终表格如下：

	SNO	CNO	GRADE	SEMESTER	TEACHINGCLASS
1	20180001	81001	85	20192	81001-01
2	20180001	81002	96	20201	81002-01
3	20180001	81003	87	20202	81003-01
4	20180002	81001	80	20192	81001-02
5	20180002	81002	98	20201	81002-01
6	20180002	81003	71	20202	81003-02
7	20180003	81001	81	20192	81001-01
8	20180003	81002	76	20201	81002-02
9	20180004	81001	56	20192	81001-02
10	20180004	81002	97	20201	81002-02
11	20180005	81003	68	20202	81003-01

2、SQL 查询

（1）单表查询——查询计算机科学与技术和数据科学与大数据技术的学生姓名、出生日期和主修专业

```sql
#1、查询计算机科学与技术和数据科学与大数据技术的学生姓名、出生日期和主修专业
SELECT SNAME,SNO,SMAJOR
FROM student
WHERE SMAJOR IN ('计算机科学与技术','数据科学与大数据技术');
```

结果如下：

	SNAME	SNO	SMAJOR
1	刘晨	20180002	计算机科学与技术
2	王敏	20180003	计算机科学与技术
3	张立	20180004	计算机科学与技术
4	赵明	20180006	数据科学与大数据技
5	王佳佳	20180007	数据科学与大数据技

（2）多表查询——查询选修了 81003 课程的学生的学号、名字及其成绩，查询结果按分数排列

```
#5、查询选修了81003课程的学生的学号、名字及其成绩，查询结果按分数排列
SELECT student.SNO, student.SNAME, SC.GRADE
FROM student, SC
WHERE CNO = '81003'
ORDER BY GRADE DESC ;
```

查询结果如下：

	SNO	SNAME	GRADE
1	20180001	李勇	87
2	20180002	刘晨	71
3	20180005	陈新奇	68

（3）字符匹配——查询课程名称为 DB_开头，且倒数第三个字符为 i 的课程名称和课程号

在 MySQL 中，不需要使用 ESCAPE，可以直接在 like 之后使用转义字符\。

```
#4、查询课程名称为DB_开头，且倒数第三个字符为i的课程名称和课程号
SELECT CNAME,CNO
FROM COURSE
WHERE CNAME LIKE 'DB\_%i__' ;
```

查询结果如下：（不存在该课程）

CNAME	CNO

（4）带函数表达式的查询——查询全体学生的姓名及其年龄

使用了 extract 函数，来获取日期的年份，进行年龄的计算，并命名为“年龄”。

```
#3、查询全体学生的姓名及其年龄
SELECT SNAME,
       (extract(year from current_date) - extract(year from SBIRTHDATE)) "年龄"
FROM student
```

查询结果如下：

	SNAME	年龄
1	李勇	25
2	刘晨	26
3	王敏	24
4	张立	25
5	陈新奇	24
6	赵明	25
7	王佳佳	24

（5）GROUP BY 和 HAVING 子句——查询选修课程数超过 2 门的学生学号

WHERE 作用于基本表或视图，HAVING 作用于元组；

GROUP BY 对查询结果分组，HAVING 对查询结果筛选。

```
#5、查询选修课程数超过2门的学生学号
SELECT SNO
FROM SC
GROUP BY SNO
HAVING COUNT(*) > 2;
```

查询结果如下：

	SNO
1	20180001
2	20180002

（6）相关子查询——找出每个学生超过他自己选修课程的平均成绩的课程号

```
#6、找出每个学生超过他自己选修课程的平均成绩的课程号
SELECT SNO,CNO
FROM SC X
WHERE GRADE >= (SELECT AVG(GRADE)
                FROM SC Y
                WHERE Y.SNO = X.SNO);
```

查询结果如下：

	SNO	CNO
1	20180001	81002
2	20180002	81002
3	20180003	81001
4	20180004	81002
5	20180005	81003

（7）不相关子查询——查询于“刘晨”在同一个专业的学生学号、姓名、主修专业

```
#7、查询与“刘晨”在同一个专业的学生学号、姓名、主修专业
SELECT SNO,SNAME,SMAJOR
FROM student
WHERE SMAJOR IN
    (SELECT SMAJOR
     FROM student
     WHERE SNAME = '刘晨');
```

查询结果如下：

	SNO	SNAME	SMAJOR
1	20180002	刘晨	计算机科学与技术
2	20180003	王敏	计算机科学与技术
3	20180004	张立	计算机科学与技术

（8）集函数查询——查询选修了课程的学生人数

因为学生人数不可重复计算，所以需要对结果进行去重

```
#8、查询选修了课程的学生人数
SELECT COUNT(DISTINCT SNO)
FROM SC;
```

查询结果如下：

	`COUNT(DISTINCT SNO)`
1	5

（9）表达“全部”语义的查询——查询选修了 81001 课程的全部学生

```sql
#9、查询选修了81001课程的全部学生
SELECT SNO,SNAME
FROM student
WHERE EXISTS
    (SELECT *
     FROM SC
     WHERE CNO = '81001');
```

查询结果如下：

	SNO	SNAME
1	20180001	李勇
2	20180002	刘晨
3	20180003	王敏
4	20180004	张立
5	20180005	陈新奇
6	20180006	赵明
7	20180007	王佳佳

（10）表达“至少”语义的查询——查询代至少选修了学生 20180002 选修的全部课程的学生的学号

p：学生 20180002 选修了课程 y

q：学生 x 选修了课程 y

$(\forall y)\ p \rightarrow q \equiv \neg(\exists y(\neg(p \rightarrow q))) \equiv \neg(\exists y(\neg(\neg p \vee q))) \equiv \neg \exists y(p \wedge \neg q)$

```sql
#10、查询代至少选修了学生20180002选修的全部课程的学生的学号
SELECT SNO
FROM student
WHERE NOT EXISTS
    (SELECT *
     FROM SC X
     WHERE X.SNO = '20180002' AND
           NOT EXISTS
               (SELECT *
                FROM Y
                WHERE Y.SNO = student.SNO AND Y.CNO = X.CNO));
```

查询结果如下：

	SNO
1	20180001
2	20180002

*第二种做法：利用 NOT EXISTS + EXCEPT 反向验证需求
子查询 1：20180002 所选的全部课程
子查询 2：当前要验证的学生的全部选课
EXCEPT：左集合 - 右集合，只保留左有右没有
逻辑：对每个学生 x，判断（20180002 选课 - x 选课）是否为空集，若是说明 X 选了 20180002 的所有课；如果不为空，说明 X 缺了 20180002 的某些课。

```
SELECT DISTINCT SNO
FROM SC X
WHERE NOT EXISTS
    (SELECT CNO FROM SC Y WHERE Y.SNO = '20180002')
    EXCEPT
    (SELECT CNO FROM SC Z WHERE Z.SNO = X.SNO)
```

（11）带 EXIST 的子查询——查询所有没有选修 81002 号课程的学生姓名
先查询所有选修 81002 课程的学生，然后找出没有选的。

```
#11、查询所有没有选修81002号课程的学生姓名
SELECT SNAME
FROM student
WHERE NOT EXISTS
    (SELECT *
     FROM SC
     WHERE SNO = student.SNO AND CNO = '81002');
```

查询结果如下：

	SNAME
1	陈新奇
2	赵明
3	王佳佳

（12）使用子查询进行批量插入，删除以及修改语句——将计算机科学与技术的学生成绩置为 0

```sql
#12、将计算机科学与技术的学生成绩置为0
UPDATE SC
SET GRADE = 0
WHERE SNO IN
      (SELECT SNO
       FROM student
       WHERE SMAJOR = '计算机科学与技术');
```

查询结果如下：

	SNO	CNO	GRADE	SEMESTER	TEACHINGCLASS
1	20180005	81003	68	20202	81003-01
2	20180004	81002	0	20201	81002-02
3	20180004	81001	0	20192	81001-02
4	20180003	81002	0	20201	81002-02
5	20180003	81001	0	20192	81001-01
6	20180002	81003	0	20202	81003-02
7	20180002	81002	0	20201	81002-01
8	20180002	81001	0	20192	81001-02
9	20180001	81003	87	20202	81003-01
10	20180001	81002	96	20201	81002-01
11	20180001	81001	85	20192	81001-01

（13）自身链接——查询每一门课的间接先修课（先修课的先修课）

自身链接一定要给表起别名来区分。

```sql
#13、查询每一门课的间接先修课（先修课的先修课）
SELECT FIRST.CNO,SECOND.CPNO '间接先修'
FROM COURSE FIRST, COURSE SECOND
WHERE FIRST.CPNO = SECOND.CNO AND SECOND.CPNO IS NOT NULL ;
```

查询结果如下：

	CNO	间接先修
1	81003	81001
2	81006	81001
3	81004	81002
4	81008	81002

（14）外连接查询 —— 查询每个学生的学号姓名以及选课情况

```sql
#14、查询每个学生的学号姓名以及选课情况
SELECT STU.SNO,SNAME,CNO,GRADE,SEMESTER
FROM student STU LEFT OUTER JOIN SC S on STU.SNO = S.SNO;
```

查询结果如下：

	SNO	SNAME	CNO	GRADE	SEMESTER
1	20180001	李勇	81001	85	20192
2	20180001	李勇	81002	96	20201
3	20180001	李勇	81003	87	20202
4	20180002	刘晨	81001	80	20192
5	20180002	刘晨	81002	98	20201
6	20180002	刘晨	81003	71	20202
7	20180003	王敏	81001	81	20192
8	20180003	王敏	81002	76	20201
9	20180004	张立	81001	56	20192
10	20180004	张立	81002	97	20201
11	20180005	陈新奇	81003	68	20202
12	20180006	赵明	<null>	<null>	<null>
13	20180007	王佳佳	<null>	<null>	<null>

（15）集合查询——查询即选修 81001 课程又选修 81002 课程的学生

```
#15、查询即选修81001课程又选修81002课程的学生
SELECT SC.SNO,SNAME
FROM SC,student
WHERE SC.SNO = student.SNO AND CNO = '81001'
INTERSECT
SELECT SC.SNO,SNAME
FROM SC,student
WHERE SC.SNO = student.SNO AND CNO = '81002'
```

查询结果如下：

	SNO	SNAME
1	20180001	李勇
2	20180002	刘晨
3	20180003	王敏
4	20180004	张立

（16）基于派生表的查询——找出每个学生超过他自己选修课程平均成绩的课程号

```
#16、找出每个学生超过他自己选修课程平均成绩的课程号
SELECT SNO,CNO
FROM SC,
     (SELECT SNO,AVG(GRADE) FROM SC GROUP BY SNO)
    AS AVG_SC(AVG_SNO,AVG_GRADE)
WHERE SC.SNO = AVG_SC.AVG_SNO AND GRADE >= AVG_GRADE;
```

查询结果如下：

	SNO	CNO
1	20180001	81002
2	20180002	81002
3	20180003	81001
4	20180004	81002
5	20180005	81003

（17）集合查询和 exists 转换——查询选修选修课程 81001 和 81002 课程的学生

```
#17、查询选修选修课程81001和81002课程的学生（用exist改写）
SELECT SNO
FROM SC X
WHERE X.CNO = '81001' AND EXISTS
    (SELECT *
     FROM SC Y
     WHERE X.SNO = Y. SNO AND Y.CNO = '81002');
```

查询结果如下：

	SNO
1	20180001
2	20180002
3	20180003
4	20180004

（18）带有 ANY 谓词的查询——查询其他戏中比计算机科学与技术某一学生年龄都小的学生姓名和年龄

```
#18、查询其他戏中比计算机科学与技术某一学生年龄都小的学生姓名和年龄
SELECT  SNAME,
        (EXTRACT(year from current_date) - EXTRACT(year from SBIRTHDATE)) "年龄"
FROM student
WHERE SBIRTHDATE < ANY (SELECT SBIRTHDATE
                        FROM student
                        WHERE SMAJOR = '计算机科学与技术');
```

查询结果如下：

	SNAME	年龄
1	李勇	25
2	刘晨	26
3	张立	25
4	赵明	25

（19）带子查询的 HAVING 子句 ——查询每个学生的最高成绩及其对应的课程名称

```sql
#19、查询每个学生的最高成绩及其对应的课程名称
SELECT S.SNO,SNAME,CNAME,MAX(GRADE) AS highest_grade
FROM SC X
JOIN COURSE C ON C.CNO = X.CNO
JOIN student S ON X.SNO = S.SNO
GROUP BY S.SNO,C.CNAME
HAVING MAX(GRADE) = (
    SELECT MAX(GRADE)
    FROM SC Y
    WHERE Y.SNO = S.SNO
    )
```

查询结果如下：

	SNO	SNAME	CNAME	highest_grade
1	20180001	李勇	数据结构	96
2	20180002	刘晨	数据结构	98
3	20180003	王敏	程序设计基础与C	81
4	20180004	张立	数据结构	97
5	20180005	陈新奇	数据库系统概论	68

（20）带有条件表达式的查询——查询每门课程获得 80 分以上成绩的概率

CASE WHEN 条件 THEN 结果 [ELSE 结果] END（ELSE 可省略，默认返回 NULL）

```sql
#20、查询每门课程获得80分以上成绩的概率
SELECT CNO,
       COUNT(CASE WHEN GRADE >= 80 THEN 1 END) * 100.0 / COUNT(*)
        AS COURSE_RATE
FROM SC
GROUP BY CNO;
```

查询结果如下：

	CNO	COURSE_RATE
1	81001	75.00000
2	81002	75.00000
3	81003	33.33333

三、心得总结（写出自己在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

（一）问题和解决方法

1、在插入 course 表数据时，由于不是每门课都有先修课，因此有些实例的 CPNO 属性值为空，但在运行 SQL 代码，发生错误，错误代码以及报错如下：

```
INSERT INTO COURSE
VALUES ('81001','程序设计基础与C语言','4',''),
       ('81002','数据结构','4','81001'),
       ('81003','数据库系统概论','4','81002'),
       ('81004','信息系统概论','4','81003'),
       ('81005','操作系统','4','81001'),
       ('81006','Python语言','3','81002'),
       ('81007','离散数学','4',''),
       ('81008','大数据技术概论','4','81003');
```

```
[23000][1452] Cannot add or update a child row: a foreign key constraint fails (`lab3`.`course`, CONSTRAINT `course_ibfk_1` FOREIGN KEY (`CPNO`) REFERENCES `course` (`CNO`) ON UPDATE CASCADE)
```

通过查询该报错原因，核心问题在于：外键约束要求 CPNO 的值必须是 CNO 中已存在的有效值，或严格为 NULL。NULL 表示“无值”或者“未知”，但 ‘’ 表示有值，但值为空，会被当作实际字符串参与外键的校验，而 CNO 中没有与 ‘’ 相同的值，因此报错。

修改后代码如下：

```
INSERT INTO COURSE
VALUES ('81001','程序设计基础与C语言','4',NULL),
       ('81002','数据结构','4','81001'),
       ('81003','数据库系统概论','4','81002'),
       ('81004','信息系统概论','4','81003'),
       ('81005','操作系统','4','81001'),
       ('81006','Python语言','3','81002'),
       ('81007','离散数学','4',NULL),
       ('81008','大数据技术概论','4','81003');
```

2、在进行多表查询时，想要查询选修某一课程号下的学生信息和成绩，发现结果变成了学生信息和成绩的笛卡尔积，成绩和学生信息没有完成一一对应。

错误代码和查询结果如下：

```
#5、查询选修了81003课程的学生的学号、名字及其成绩，查询结果按分数排列
SELECT student.SNO, student.SNAME, SC.GRADE
FROM student, SC
WHERE CNO = '81003'
ORDER BY GRADE DESC ;
```

	SNO	SNAME	GRADE
1	20180001	李勇	87
2	20180002	刘晨	87
3	20180003	王敏	87
4	20180004	张立	87
5	20180005	陈新奇	87
6	20180006	赵明	87
7	20180007	王佳佳	87
8	20180001	李勇	71
9	20180002	刘晨	71
10	20180003	王敏	71
11	20180004	张立	71
12	20180005	陈新奇	71
13	20180006	赵明	71
14	20180007	王佳佳	71
15	20180001	李勇	68
16	20180002	刘晨	68
17	20180003	王敏	68
18	20180004	张立	68

根据查询的结果，可以看到查询的语句本身没有问题，但由于在进行多表查询时，没有指定相互关联的元素，使得学生信息和成绩没法一一对应，形成了笛卡尔积，每一个成绩对应了所有的学生，修改后代码如下：

```
#5、查询选修了81003课程的学生的学号、名字及其成绩，查询结果按分数排列
SELECT student.SNO, student.SNAME, SC.GRADE
FROM student, SC
WHERE student.SNO = SC.SNO AND CNO = '81003'
ORDER BY GRADE DESC ;
```

	SNO	SNAME	GRADE
1	20180001	李勇	87
2	20180002	刘晨	71
3	20180005	陈新奇	68

（二）体会与收获

通过实际的查询操作练习，加深了对于各类查询、修改、删除等语句的使用方法，提高了

设局库设计能力，在设计数据库时学会了如何根据实际需求进行表结构设计，选择荷时的数据类型，并添加约束条件以及完整性约束，学会了如何优化表关系，从而减少数据冗余和提高查询效率。学会了在遇到问题时，如何快速排查错误原因，并查询资料来找到解决问题的方法。通过优化查询语句，来提高查询效率以及查询语句的可读性。